

M3_Regressió_Polinòmica_i_Regularització

December 15, 2025

```
[20]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.preprocessing import PolynomialFeatures
```

Carreguem les dades:

```
[21]: diabetes = load_diabetes()
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target
```

Partim de que a través de la Matriu de correlacions hem detectat els “best features” en el notebook anterior (BMI, S5, BP).

```
[22]: best_features = ['bmi', 's5', 'bp']
```

```
[23]: # Correlacions
correlations = df[best_features + ['target']].corr()['target'].drop('target')
print(f"\nCorrelacions amb target:")
for feat, corr in correlations.items():
    print(f"    {feat:6s}: {corr:+.4f}")
```

Correlacions amb target:

```
    bmi    : +0.5865
    s5     : +0.5659
    bp     : +0.4415
```

A continuació comprovarem si la relació entre aquests “features” i el target és realment lineal o no.

```
[24]: fig, axes = plt.subplots(1, 3, figsize=(20, 6))

for idx, feature in enumerate(best_features):
    ax = axes[idx]
```

```

# Scatter plot amb punts
scatter = ax.scatter(df[feature], df['target'], alpha=0.6, s=40,
                    color='steelblue', edgecolors='darkblue', linewidth=0.5)

# Línia LINEAL (grau 1) - COLOR ROIG
z1 = np.polyfit(df[feature], df['target'], 1)
p1 = np.poly1d(z1)
x_line = np.linspace(df[feature].min(), df[feature].max(), 100)
line1 = ax.plot(x_line, p1(x_line), "r--", alpha=0.9, linewidth=4,
                label=f'LINEAL')[0]

# Línia POLINÒMICA (grau 2) - COLOR VERD
z2 = np.polyfit(df[feature], df['target'], 2)
p2 = np.poly1d(z2)
line2 = ax.plot(x_line, p2(x_line), "g-", alpha=0.9, linewidth=4,
                label=f'POLINÒMICA')[0]

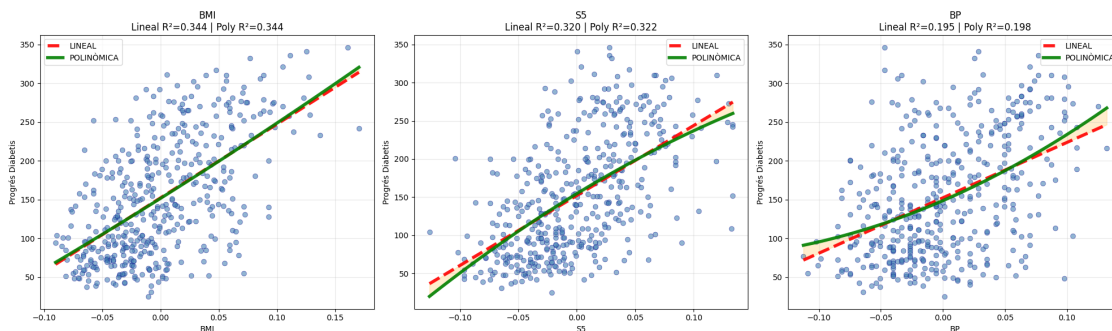
# R² per comparar QUINA és millor
r2_lineal = r2_score(df['target'], p1(df[feature]))
r2_poly = r2_score(df['target'], p2(df[feature]))

ax.set_xlabel(f'{feature.upper()}')
ax.set_ylabel('Progrés Diabetis')
ax.set_title(f'{feature.upper()}\n'
            f'Lineal R²={r2_lineal:.3f} | Poly R²={r2_poly:.3f}')
ax.legend()
ax.grid(True, alpha=0.3)

# COLORAR ÀREA entre corbes (distància = ERROR)
ax.fill_between(x_line, p1(x_line), p2(x_line),
                alpha=0.2, color='orange', label='DISTANÇA=ERROR')

plt.tight_layout()
plt.show()

```



0.1 REGRESSIÓ POLINÒMICA

La regressió polinòmica estén el model lineal afegint termes polinòmics:

MODEL LINEAL:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

MODEL POLINÒMIC (grau 2):

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4 + \beta_5 x^5 + \beta_6 x^6 + \beta_7 x^7 + \beta_8 x^8 + \beta_9 x^9 + \beta_{10} x^{10}$$

Passem de 3 “features” (en aquest cas) a combinacions entre ells:

$$[x, x^2, x^3] \rightarrow [x, x^2, x^3, x^4, x^5, x^6, x^7, x^8, x^9, x^{10}]$$

GRAUS RECOMANATS: - Grau 1: Model lineal - Grau 2: Captura curvatures simples i interaccions - RECOMANAT - Grau 3: Per relacions més complexes (perill d’overfitting) - Grau 4: Gairebé sempre overfitting

Povem el resultat de les regressions polinòmiques de diferents graus:

```
[25]: best_features = ['bmi', 's5', 'bp']
X = df[best_features]
y = df['target']

print(f"Features: {best_features}")
print(f"Shape: {X.shape}")

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

Features: ['bmi', 's5', 'bp']

Shape: (442, 3)

```
[26]: # Provar diferents graus
degrees = [1, 2, 3, 4]
results = {}

for degree in degrees:

    print(f"GRAU {degree}")

    # Crear features polinòmiques
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    n_features = X_train_poly.shape[1]
    print(f"    Features generades: {n_features}")

    if degree == 2:
        feature_names_poly = poly.get_feature_names_out(best_features)
```

```

    print(f"    Exemples: {list(feature_names_poly[:8])}")

# Entrenar model
model = LinearRegression()
model.fit(X_train_poly, y_train)

# Prediccions
y_train_pred = model.predict(X_train_poly)
y_test_pred = model.predict(X_test_poly)

# Mètriques
r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
mae_test = mean_absolute_error(y_test, y_test_pred)
gap = r2_train - r2_test

results[degree] = {
    'model': model,
    'poly': poly,
    'r2_train': r2_train,
    'r2_test': r2_test,
    'rmse_test': rmse_test,
    'mae_test': mae_test,
    'gap': gap,
    'n_features': n_features,
    'predictions': y_test_pred
}

print(f"\n    MÈTRIQUES:")
print(f"        R² train:  {r2_train:.4f}")
print(f"        R² test:   {r2_test:.4f}")
print(f"        Gap:       {gap:.4f}")
print(f"        RMSE test: {rmse_test:.2f}")
print(f"        MAE test:  {mae_test:.2f}")

```

GRAU 1

Features generades: 3

MÈTRIQUES:

R² train: 0.4831

R² test: 0.4543

Gap: 0.0288

RMSE test: 53.77

MAE test: 44.29

GRAU 2

Features generades: 9

Exemples: ['bmi', 's5', 'bp', 'bmi^2', 'bmi s5', 'bmi bp', 's5^2', 's5 bp']

MÈTRIQUES:

R² train: 0.4976
R² test: 0.4668
Gap: 0.0308
RMSE test: 53.15
MAE test: 44.03

GRAU 3

Features generades: 19

MÈTRIQUES:

R² train: 0.5146
R² test: 0.4838
Gap: 0.0308
RMSE test: 52.30
MAE test: 42.66

GRAU 4

Features generades: 34

MÈTRIQUES:

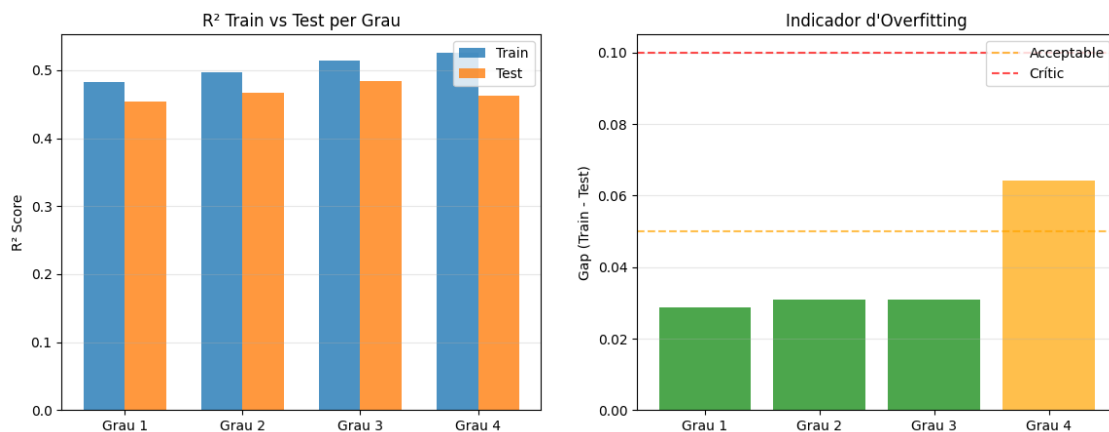
R² train: 0.5261
R² test: 0.4619
Gap: 0.0643
RMSE test: 53.39
MAE test: 43.56

```
[27]: # Evolució de mètriques
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# R² comparison
ax = axes[0]
x = [f'Grau {d}' for d in degrees]
r2_trains = [results[d]['r2_train'] for d in degrees]
r2_tests = [results[d]['r2_test'] for d in degrees]

width = 0.35
x_pos = np.arange(len(x))
ax.bar(x_pos - width/2, r2_trains, width, label='Train', alpha=0.8)
ax.bar(x_pos + width/2, r2_tests, width, label='Test', alpha=0.8)
ax.set_ylabel('R² Score')
ax.set_title('R² Train vs Test per Grau')
ax.set_xticks(x_pos)
ax.set_xticklabels(x)
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
```

```
# Gap (overfitting indicator)
ax = axes[1]
gaps = [results[d]['gap'] for d in degrees]
colors = ['green' if g < 0.05 else 'orange' if g < 0.10 else 'red' for g in gaps]
ax.bar(x, gaps, color=colors, alpha=0.7)
ax.set_ylabel('Gap (Train - Test)')
ax.set_title('Indicador d\'Overfitting')
ax.axhline(y=0.05, color='orange', linestyle='--', alpha=0.7, label='Acceptable')
ax.axhline(y=0.10, color='red', linestyle='--', alpha=0.7, label='Crític')
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
```



0.2 Regularització (Ridge, Lasso i ElasticNet)

Els Models complexos (polinòmics de grau alt) tendeixen a OVERFITTING.

RIDGE REGRESSION Molt bo en eliminar multicolinealitat (features dependents els uns dels altres).

LASSO REGRESSION Molt bo en eliminar coeficients no rellevants.

ELASTICNET Combinació de les dos anteriors.

Veguem primer l'efecte d'aplicar aquestes regressions a les dades originals per entendre una mica quin és el seu efecte:

```
[17]: from sklearn.linear_model import Lasso, Ridge, ElasticNet
      from sklearn.feature_selection import SelectFromModel
      from sklearn.preprocessing import StandardScaler
```

```
[30]: # Crear features polinòmiques
      poly = PolynomialFeatures(degree=2, include_bias=False)
```

```

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
feature_names_poly = poly.get_feature_names_out(best_features)
def optim_alpha(model_class, alphas, X_poly, y_train):
    model_with_iter = model_class(max_iter=10000)
    param_grid = {'alpha': alphas}
    grid = GridSearchCV(model_with_iter, param_grid, cv=5, scoring='r2',
↪n_jobs=-1)
    grid.fit(X_poly, y_train)
    return grid.best_params_['alpha'], grid.best_score_

alphas = [0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0]

alpha_lasso, r2_lasso = optim_alpha(Lasso, alphas, X_train_poly, y_train)
alpha_ridge, r2_ridge = optim_alpha(Ridge, alphas, X_train_poly, y_train)
alpha_elastic, r2_elastic = optim_alpha(ElasticNet, alphas, X_train_poly,
↪y_train)

print(f"Lasso:  ={alpha_lasso} (R²={r2_lasso:.3f})")
print(f"Ridge:  ={alpha_ridge} (R²={r2_ridge:.3f})")
print(f"ElasticNet:  ={alpha_elastic} (R²={r2_elastic:.3f})")

```

```

Lasso:  =0.05 (R²=0.440)
Ridge:  =0.01 (R²=0.444)
ElasticNet:  =0.01 (R²=0.305)

```

```

[31]: alphas_dict = {
        'Lasso': alpha_lasso,
        'Ridge': alpha_ridge,
        'ElasticNet': alpha_elastic
    }

models = {
    'Lasso': Lasso(alpha=alpha_lasso, max_iter=5000, random_state=42),
    'Ridge': Ridge(alpha=alpha_ridge, random_state=42),
    'ElasticNet': ElasticNet(alpha=alpha_elastic, l1_ratio=0.5, max_iter=5000,
↪random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(X_train_poly, y_train)
    coefs = model.coef_
    nonzero = sum(coefs != 0)
    results[name] = {'model': model, 'coefs': coefs, 'nonzero': nonzero}

```

```

print(f"\n{name}: {nonzero}/9 features")
coef_df = pd.DataFrame({
    'Feature': feature_names_poly,
    f'{name}_Coef': np.round(coefs, 3),
    f'{name}_Abs': np.round(np.abs(coefs), 3)
}).sort_values(f'{name}_Abs', ascending=False)
print(coef_df.to_string(index=False))

```

Lasso: 3/9 features

Feature	Lasso_Coef	Lasso_Abs
bmi	638.771	638.771
s5	480.288	480.288
bp	271.447	271.447
bmi ²	0.000	0.000
bmi s5	0.000	0.000
bmi bp	0.000	0.000
s5 ²	-0.000	0.000
s5 bp	0.000	0.000
bp ²	0.000	0.000

Ridge: 9/9 features

Feature	Ridge_Coef	Ridge_Abs
bmi	621.373	621.373
bmi ²	518.260	518.260
s5	498.182	498.182
bmi bp	428.277	428.277
bp	279.987	279.987
s5 bp	222.355	222.355
s5 ²	-188.994	188.994
bp ²	178.914	178.914
bmi s5	147.473	147.473

ElasticNet: 9/9 features

Feature	ElasticNet_Coef	ElasticNet_Abs
bmi	253.027	253.027
s5	224.635	224.635
bp	170.219	170.219
bmi ²	9.357	9.357
bmi bp	4.050	4.050
bp ²	3.551	3.551
bmi s5	2.050	2.050
s5 bp	1.887	1.887
s5 ²	0.982	0.982

```

[32]: for name, res in results.items():
      model = res['model']

```



```

# Prediccions
y_pred_train = model.predict(X_train_poly)
y_pred_test = model.predict(X_test_poly)

print(f"\nModel {name} (={alphas_dict[name]:.2f}):")
print(f"  R2 Train: {model.score(X_train_poly, y_train):.4f}")
print(f"  R2 Test: {r2_score(y_test, y_pred_test):.4f}")
print(f"  MSE Test: {mean_squared_error(y_test, y_pred_test):.2f}")
print(f"  MAE Test: {mean_absolute_error(y_test, y_pred_test):.2f}")
print(f"  Features: {res['nonzero']}/9 ({res['nonzero']/9*100:.0f}%)")

```

Model Lasso (=0.05):

R² Train: 0.4828
 R² Test: 0.4557
 MSE Test: 2883.97
 MAE Test: 44.29
 Features: 3/9 (33%)

Model Ridge (=0.01):

R² Train: 0.4897
 R² Test: 0.4641
 MSE Test: 2839.48
 MAE Test: 44.00
 Features: 9/9 (100%)

Model ElasticNet (=0.01):

R² Train: 0.3345
 R² Test: 0.3294
 MSE Test: 3553.04
 MAE Test: 51.12
 Features: 9/9 (100%)